

Givebutter Integration Design Document

Givebutter Integration Design Document

Version: 1.0

Date: July 8, 2026

Scope: High-level product and technical design for a Web UI, iOS app, and Android app that integrate with Givebutter CRM, fundraising, transaction, campaign, ticket, household, webhook, and widget capabilities.

1. Executive Summary

Build a cross-platform donor/fundraising experience with three clients:

- **Web UI** for staff/admin operations, dashboards, donor CRM review, campaign setup, webhook monitoring, and sync diagnostics.
- **iOS app** for donor-facing engagement, campaign discovery, donation flows, ticket access, profile updates, and push notifications.
- **Android app** with equivalent donor-facing functionality.

All Givebutter API calls should be made through a secure backend integration layer. The apps should never store or call Givebutter directly with the organization API key. Givebutter currently uses HTTPS + Bearer token authentication, has a documented <https://api.givebutter.com/v1> API surface, paginated list endpoints, a 500 requests/minute rate limit, and webhooks for important events such as campaigns, tickets, transactions, contacts, plans, and refunds.

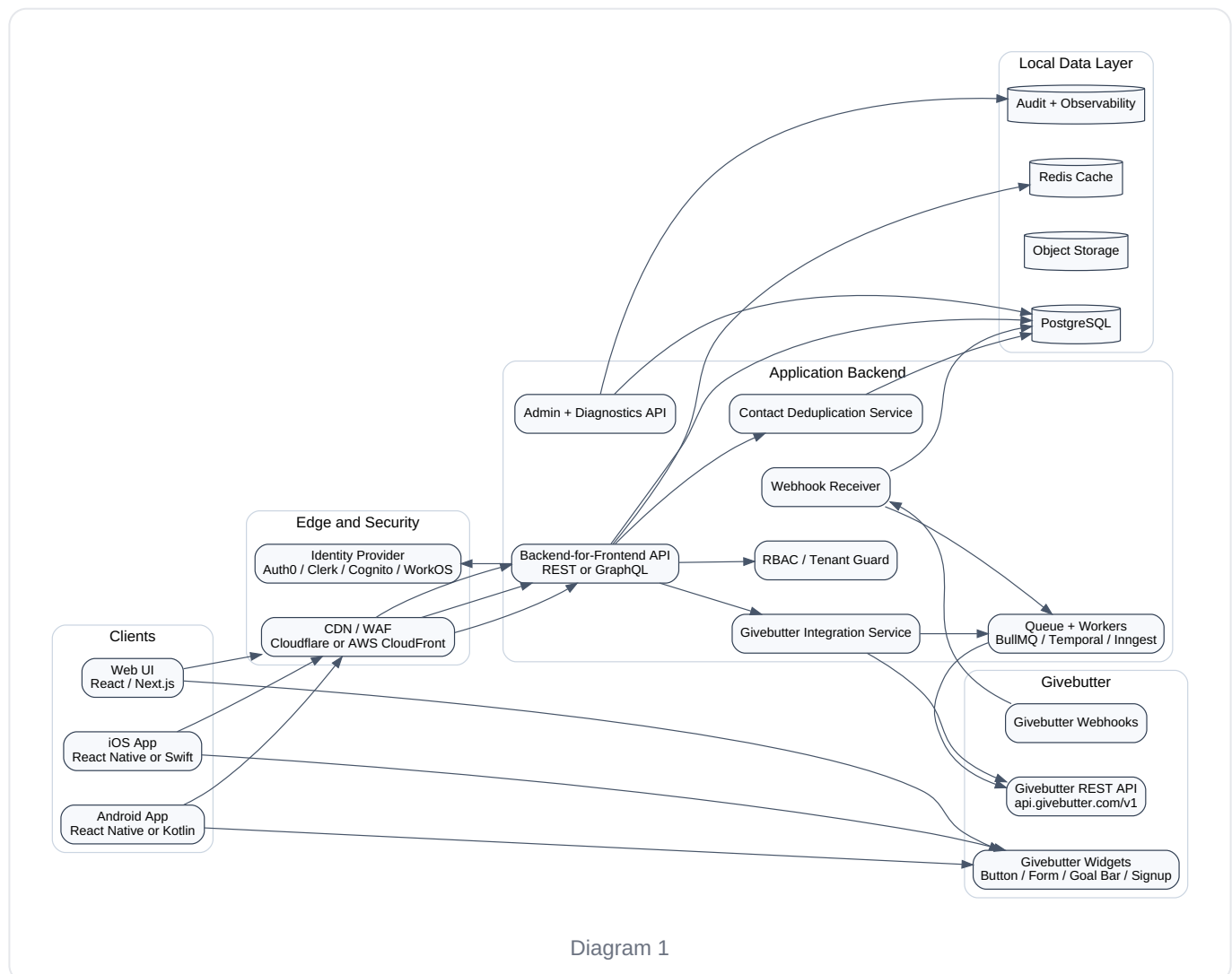
2. Goals

1. Provide a unified Web, iOS, and Android experience that maps to Givebutter CRM and fundraising data.
 2. Safely integrate with Givebutter APIs without exposing the Givebutter API key to browsers or mobile apps.
 3. Support two-way operational workflows where appropriate: create/update contacts, campaigns, funds, households, activities, transactions, webhooks, and related objects.
 4. Support webhook-driven synchronization from Givebutter into the local platform.
 5. Provide contact de-duplication before creating records and during imports/syncs.
 6. Provide auditability, observability, retry behavior, and manual review for uncertain matches or failed syncs.
-

3. Non-Goals

- Replacing Givebutter as the system of record for donations, campaigns, transactions, or fundraising workflows.
- Storing card data or processing payments directly unless separately scoped with PCI-compliant architecture.
- Exposing Givebutter API keys in client-side code.
- Assuming Givebutter supports API-level contact merging. The public API inventory includes contact create/update/delete/restore and tag/activity operations, but not a merge-contact endpoint.

4. High-Level Architecture



5. Recommended Integration Pattern

5.1 Client-to-Backend

Clients call only the platform backend:

- GET /api/campaigns
- GET /api/contacts/:id
- POST /api/contacts
- POST /api/donations/start
- GET /api/transactions
- GET /api/sync/status

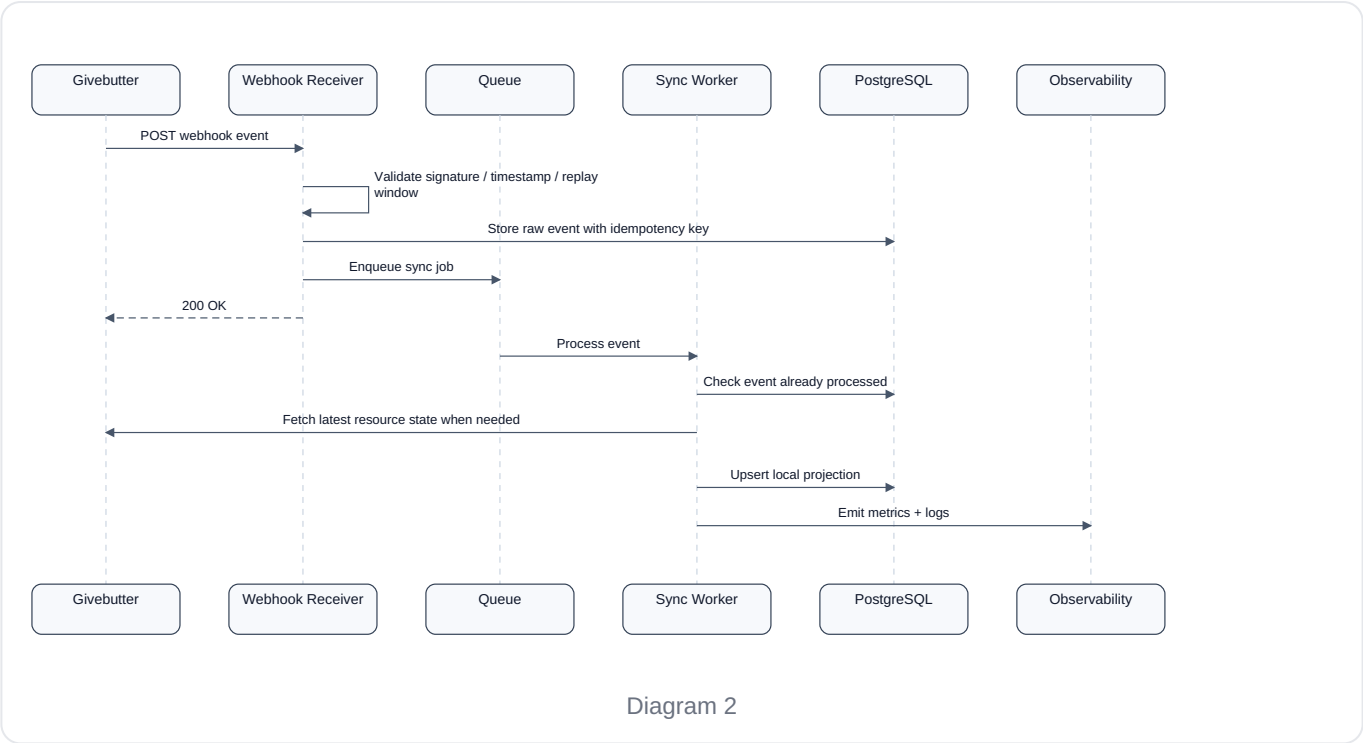
The backend enforces authentication, authorization, validation, rate limiting, audit logging, and deduplication.

5.2 Backend-to-Givebutter

The Givebutter Integration Service owns:

- API key retrieval from secret manager.
- Bearer token injection.
- Retry and backoff.
- Handling 429 Too Many Requests using the Retry-After header.
- Pagination handling.
- Mapping Givebutter resources into local DTOs.
- Logging sync attempts and failures.

5.3 Webhook-Driven Sync



Webhook processing should be idempotent. The receiver should return quickly after validation and enqueue work rather than performing long-running sync logic inline.

6. Product Surface Mapping

Product Surface	Primary Users	Givebutter Mapping	Notes
Staff Web Dashboard	Admins, fundraisers, finance	Campaigns, contacts, transactions, funds, payouts, pledges, recurring plans, webhooks	Best place for full CRM, reporting, sync diagnostics, and duplicate review.
Donor Web Portal	Donors/supporters	Contacts, contact activities, transactions, recurring plans, tickets	Should expose only authenticated donor-specific data.
iOS App	Donors/supporters, event attendees	Campaigns, tickets, transactions, donation widgets/deep links	Use backend APIs and widget/deep-link donation flows. Do not expose Givebutter API keys.
Android App	Donors/supporters, event attendees	Same as iOS	Match iOS capabilities.
Admin Sync Console	Internal operators	Webhook activities, webhooks, integration logs	Shows API health, rate-limit state, failures, retries, dedupe queue.

7. Givebutter API Inventory

The following inventory is based on Givebutter's public API documentation index as of this document date.

7.1 Platform Basics

Area	Documented Capability
Authentication	Bearer token API key authentication over HTTPS.
Pagination	Paginated list responses with <code>data</code> , <code>links</code> , and <code>meta</code> ; requests over 100 items per page return an error.
Rate limits	500 requests per minute; <code>429</code> responses include <code>Retry-After</code> .
Errors	Standard HTTP status codes, including 400, 401, 403, 404, 405, 409, 422, 429, and 5xx errors.

7.2 REST API Endpoint Groups

API Group	Available Operations
Campaign Discount Codes	Create, delete, get, list, update discount codes
Campaign Members	Delete, get, list campaign members
Campaign Teams	Delete, get, list campaign teams
Campaign Tickets	Create, get, list campaign tickets
Campaigns	Create, delete, get, list, update campaigns
Contact Activities	Create, delete, get, list, update contact activities
Contact Tags	Add tags, remove tags, sync tags for a contact
Contacts	Create, delete, get, list, restore, update contacts
Funds	Create, delete, get, list, update funds
Household Contacts	Add contact to household, get contact associated with household, list household contacts, remove contact from household
Households	Create, delete, get, list, update households
Messages	Get, list messages
Payouts	Get, list payouts
Pledges	Get, list pledges
Recurring Plans	Get, list recurring plans

API Group	Available Operations
Tickets	Get, list tickets
Transactions	Create, get, list, update transactions
Webhook Activities	Get, list webhook activities
Webhooks	Create, delete, get, list, update webhooks

7.3 Widgets and Website Integrations

Widget / Integration Area	Use Case
Button Widget	Open donation popup from web or mobile webview/deep link.
Form Widget	Embed full donation form on web pages.
Goal Bar Widget	Display campaign fundraising progress.
Signup Form Widget	Capture email subscribers.
Analytics & Attribution	Track where donations originate.
URL Prefill Parameters	Pre-fill donation amount and frequency.
Website Builder Guides	WordPress, Wix, Squarespace, Weebly, GoDaddy embedding guidance.

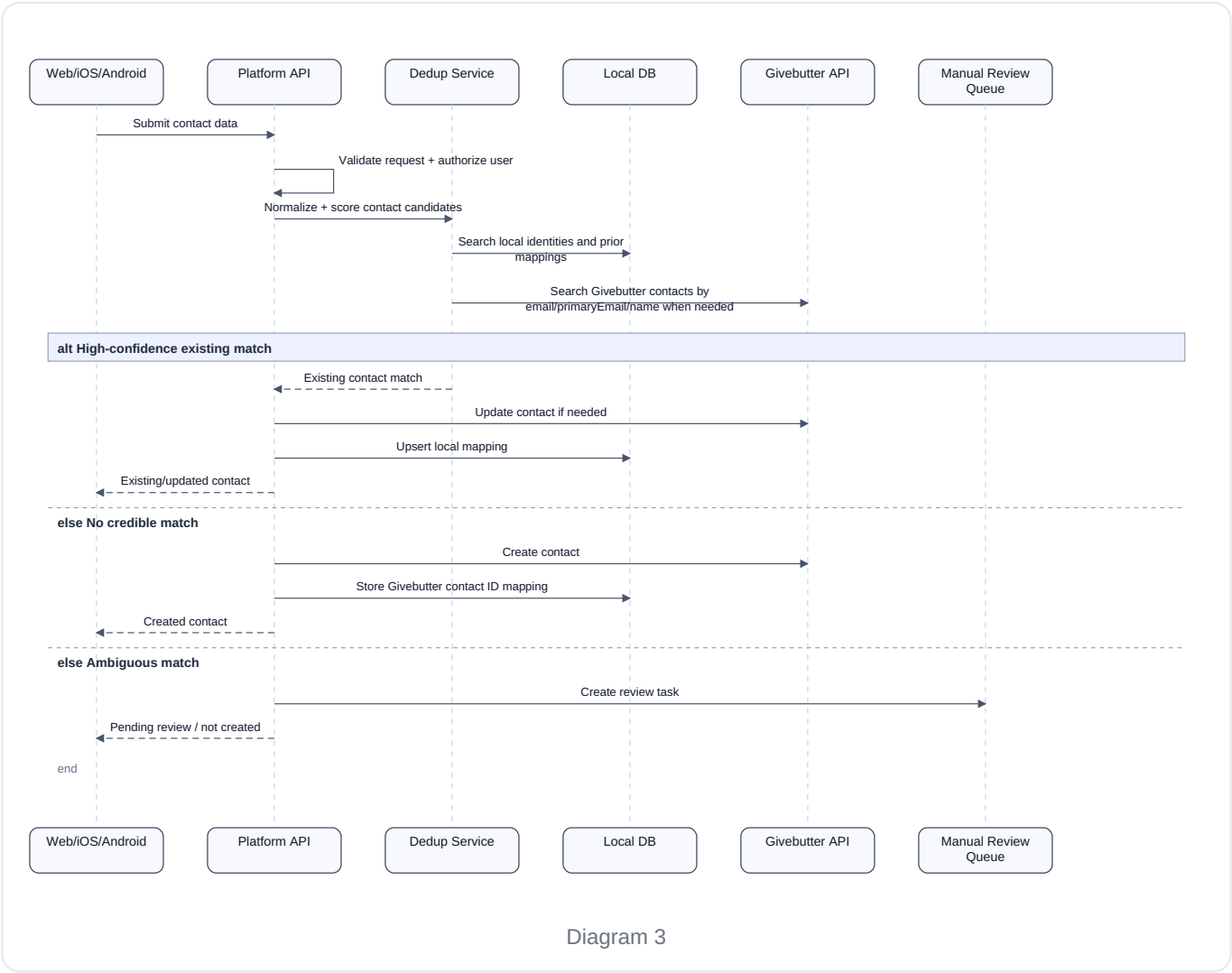
7.4 Webhook Events

Givebutter webhook creation/update supports these documented events:

- `campaign.created`
 - `campaign.updated`
 - `ticket.created`
 - `transaction.succeeded`
 - `contact.created`
 - `plan.canceled`
 - `plan.created`
 - `plan.failed`
 - `plan.paused`
 - `plan.resumed`
 - `plan.updated`
 - `refund.created`
-

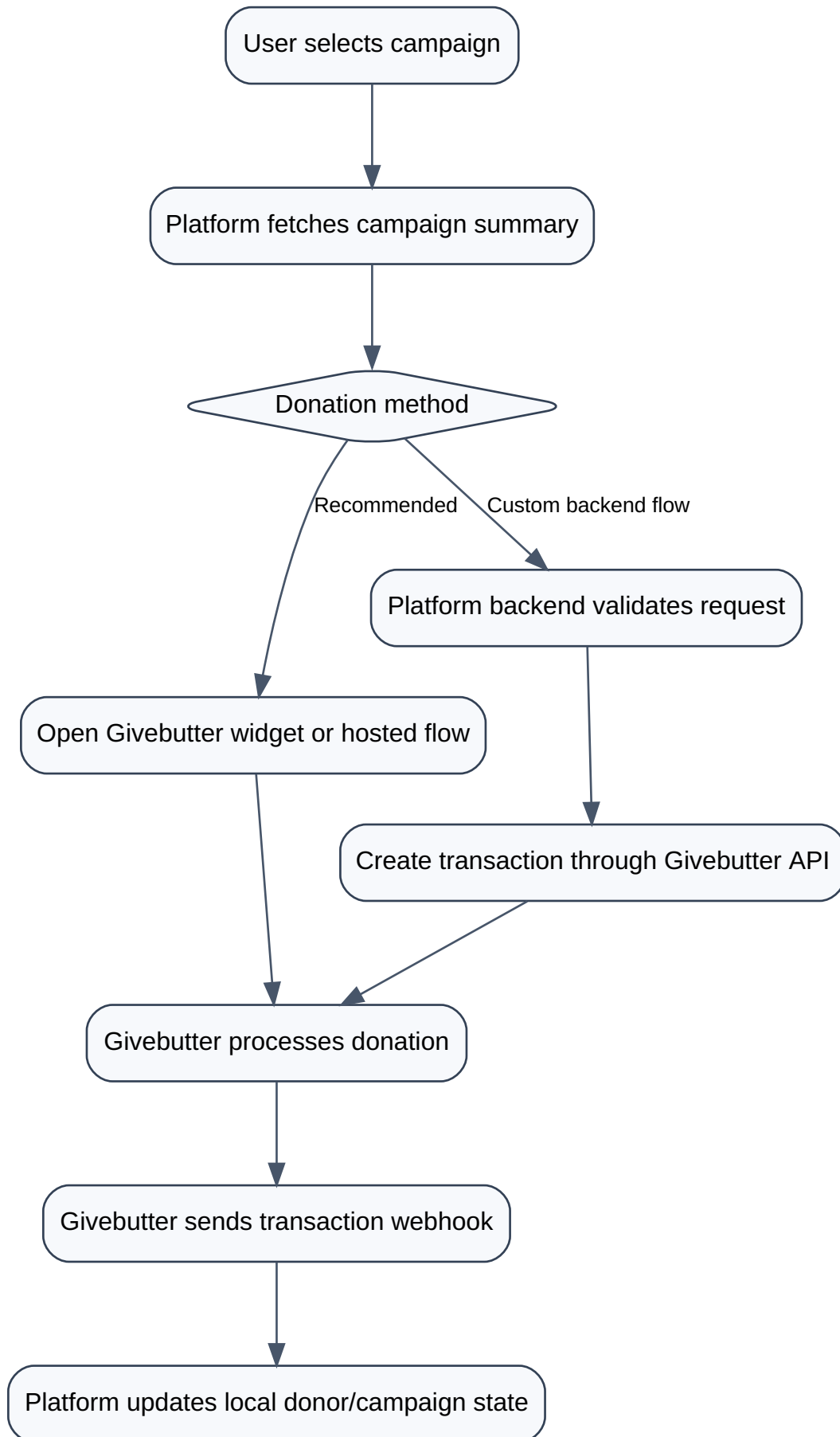
8. Core Data Flows

8.1 Contact Create / Update With Deduplication



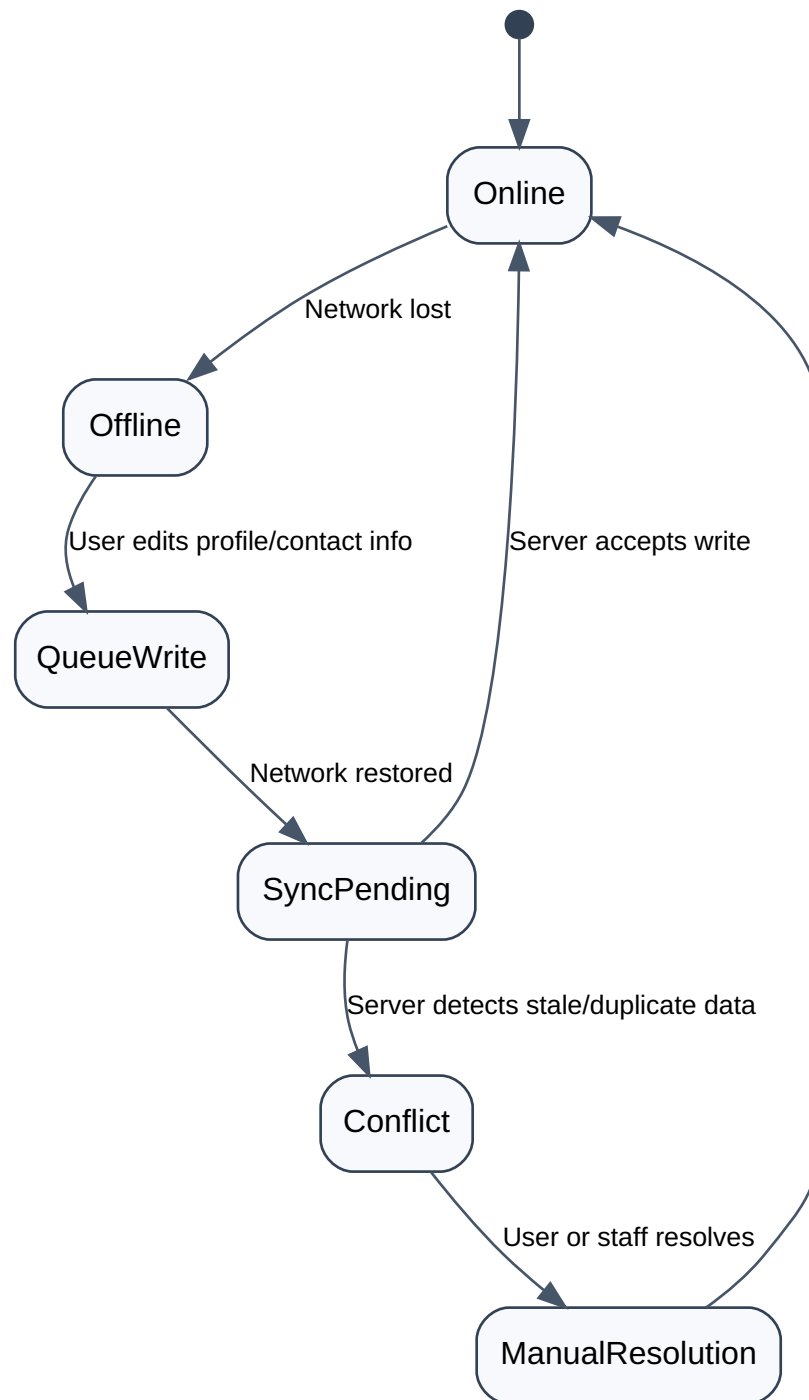
8.2 Donation Flow

Recommended approach: use Givebutter widgets or Givebutter-hosted donation experiences where possible. If a custom transaction workflow is required, route it through the backend and ensure legal, compliance, and payment scope are reviewed.



8.3 Offline Mobile Behavior

Mobile apps should cache read-only campaign and ticket views locally, but writes should be queued and retried only after re-authentication and server-side validation.



9. Recommended Tech Stack

9.1 Preferred Stack

Layer	Recommendation	Why
Web UI	Next.js + TypeScript + React Query/ TanStack Query	Strong SSR/static support, fast dashboard development, good auth integration.
Mobile	React Native + Expo + TypeScript	Shared app logic across iOS and Android; faster delivery than separate native apps.
Backend API	NestJS or Fastify + TypeScript	Good structure for integration services, validation, queues, and OpenAPI docs.
API Style	REST for external clients; optional GraphQL for dashboard aggregation	REST is simpler for mobile and web; GraphQL can reduce dashboard over-fetching.
Database	PostgreSQL	Strong relational fit for contacts, mappings, audit logs, transactions, and webhook events.
ORM	Prisma or Drizzle	Type-safe schema migrations and data access.
Queue	BullMQ + Redis for simple workloads; Temporal for complex durable workflows	Needed for webhook processing, retries, imports, and sync jobs.
Cache	Redis	API response cache, rate-limit tracking, idempotency locks.
Object Storage	S3-compatible storage	CSV imports, export files, webhook payload archives if needed.
Observability	OpenTelemetry + Sentry + Datadog/ New Relic/Grafana	Trace API calls, failed syncs, retries, and webhook processing.
CI/CD	GitHub Actions	Lint, test, typecheck, mobile build checks, infrastructure checks.

9.2 Hosting Options

Hosting Area	Practical Option	Enterprise Option
Web UI	Vercel or Cloudflare Pages	AWS CloudFront + S3 + ECS/EKS backend
Backend API	Render/Fly.io/Railway/AWS App Runner	AWS ECS Fargate, EKS, or Lambda/ API Gateway

Hosting Area	Practical Option	Enterprise Option
Database	Neon/Supabase/RDS	AWS RDS PostgreSQL Multi-AZ
Redis	Upstash/Redis Cloud	AWS ElastiCache
Secrets	Platform secrets	AWS Secrets Manager or HashiCorp Vault
Mobile CI	Expo EAS Build	Bitrise, GitHub Actions + fastlane, or EAS Enterprise

10. Security Architecture

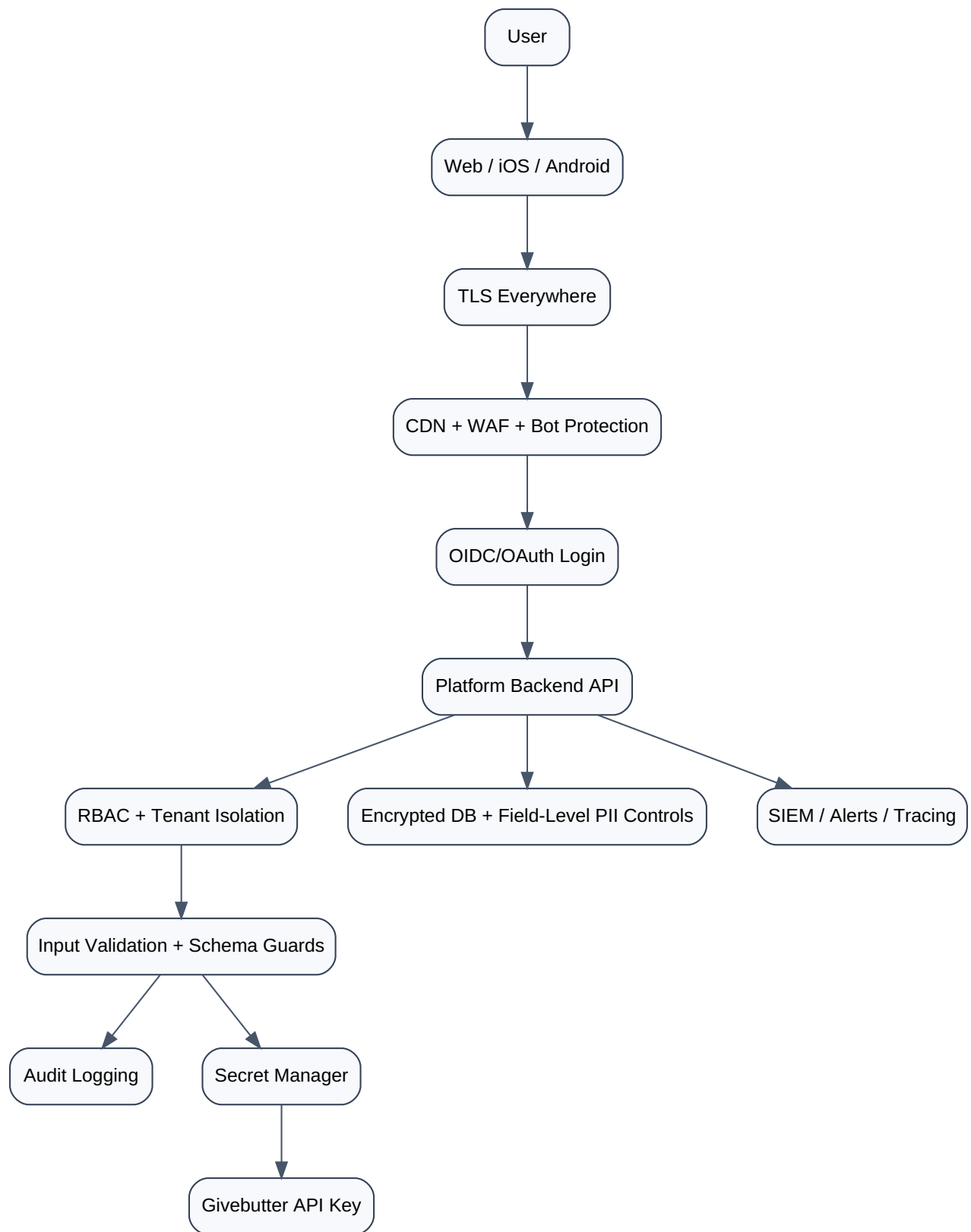


Diagram 6

10.1 Security Layers

1. Client security

- No Givebutter API key in web, iOS, or Android clients.
- Use short-lived user sessions or access tokens.
- Store only platform user tokens in secure storage on mobile.
- Apply mobile app integrity checks where justified.

2. Edge security

- CDN/WAF in front of backend.
- Rate limiting per IP, user, tenant, and route.
- Bot detection for public forms.
- Strict TLS and HSTS.

3. Identity and authorization

- OIDC provider such as Auth0, Clerk, Cognito, or WorkOS.
- RBAC roles: `donor`, `staff`, `campaign_manager`, `finance`, `admin`, `integration_admin`.
- Tenant/org isolation on every query.

4. Backend controls

- Request validation with Zod/class-validator/JSON Schema.
- Central Givebutter client with retry, timeout, pagination, and error mapping.
- Idempotency keys for creates/imports/webhook processing.
- Audit logs for contact, transaction, campaign, and admin actions.

5. Secrets and data protection

- Store Givebutter API key in Secrets Manager/Vault only.
- Rotate keys and separate production/staging keys.
- Encrypt databases at rest.
- Consider field-level encryption for sensitive donor PII.
- Avoid logging API keys, full addresses, phone numbers, or unnecessary payloads.

6. Webhook security

- Verify webhook authenticity using Givebutter's documented signature mechanism before processing.
- Store raw events with a content hash and processing state.
- Reject replayed events using timestamp/event ID/idempotency checks.
- Process asynchronously through a queue.

7. Operational security

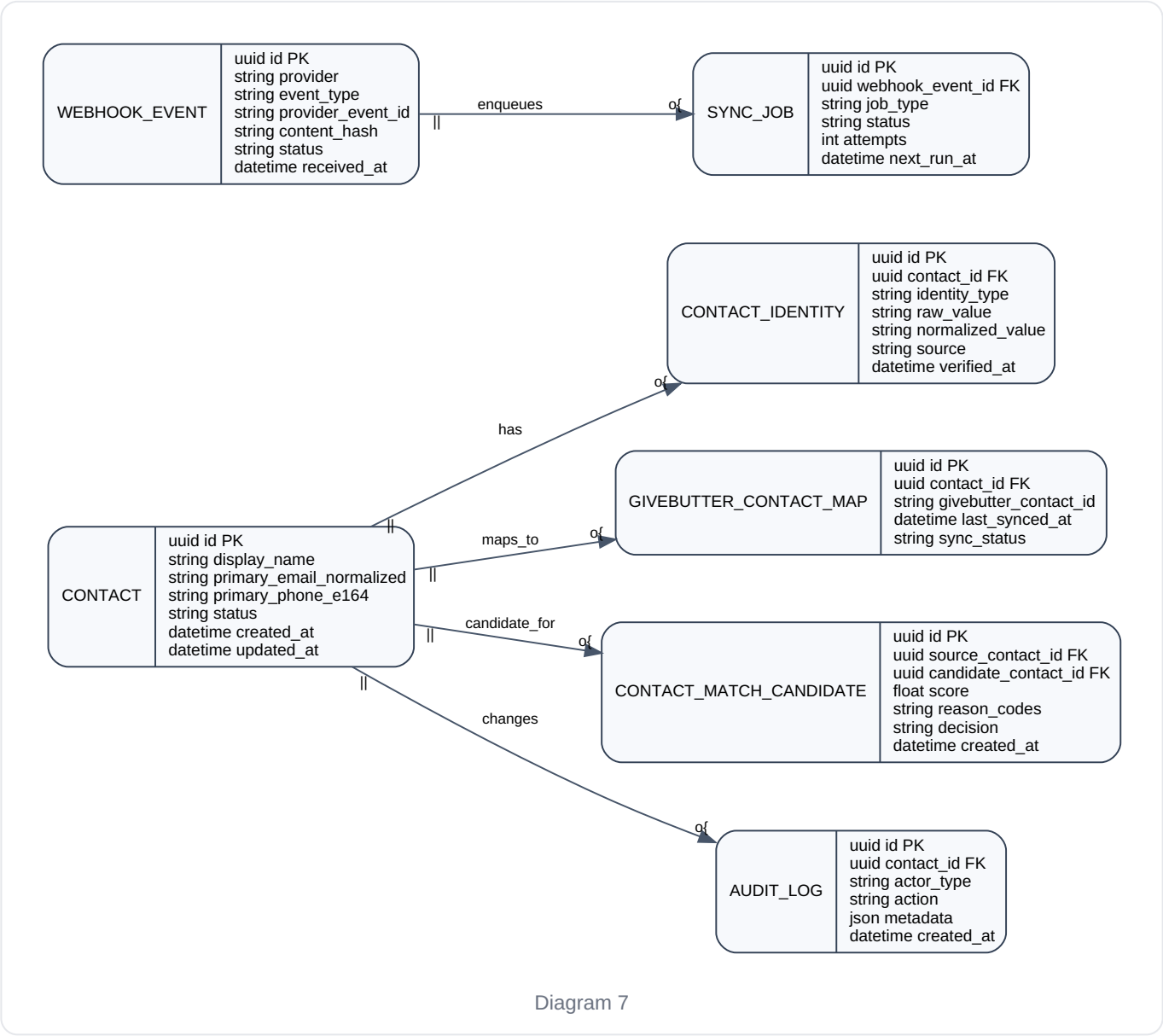
- Alert on repeated 401/403, 429, 5xx, webhook validation failures, and sync drift.
 - Run dependency scanning and secret scanning in CI.
 - Restrict admin sync tools behind elevated roles and audit logging.
-

11. Contact De-Duplication Strategy

11.1 Dedup Principle

Do not blindly create Givebutter contacts from app submissions, imports, or external systems. Every create path should first normalize the input, search local records, search Givebutter where appropriate, score candidates, and either update/link/create or route to manual review.

11.2 Recommended Local Tables



11.3 Matching Tiers

	Tier	Match Type	Suggested Action
	1	Exact verified email match	Link/update existing contact.
	2	Exact normalized phone match + compatible name	Link/update existing contact unless conflict exists.
	3	Givebutter email or primaryEmail search finds one contact	Link to Givebutter ID and update local projection.
	4	First name + last name + address/ZIP/date-of-birth-style private identifier	Score and route to review unless very high confidence.
	5	Fuzzy name + partial address/phone/email similarity	Manual review only.
	6	Household-level relationship match	Suggest household association, not automatic person merge.

11.4 Normalization Rules

- **Email:** trim whitespace, lowercase domain and local part, reject invalid syntax, preserve original value for display. Avoid aggressive provider-specific transformations unless explicitly configured.
- **Phone:** parse and store E.164 using region-aware validation.
- **Name:** trim, collapse whitespace, lowercase for matching, remove honorifics/suffixes into separate fields where possible.
- **Address:** normalize using a postal-address service such as USPS, Smarty, or Google Address Validation if address matching is important.
- **Tags:** standardize tag casing and spacing before syncing to Givebutter contact tags.
- **Source priority:** prefer verified user account data over imported CSV data; prefer Givebutter system-of-record IDs over inferred matches.

11.5 Create Contact Algorithm

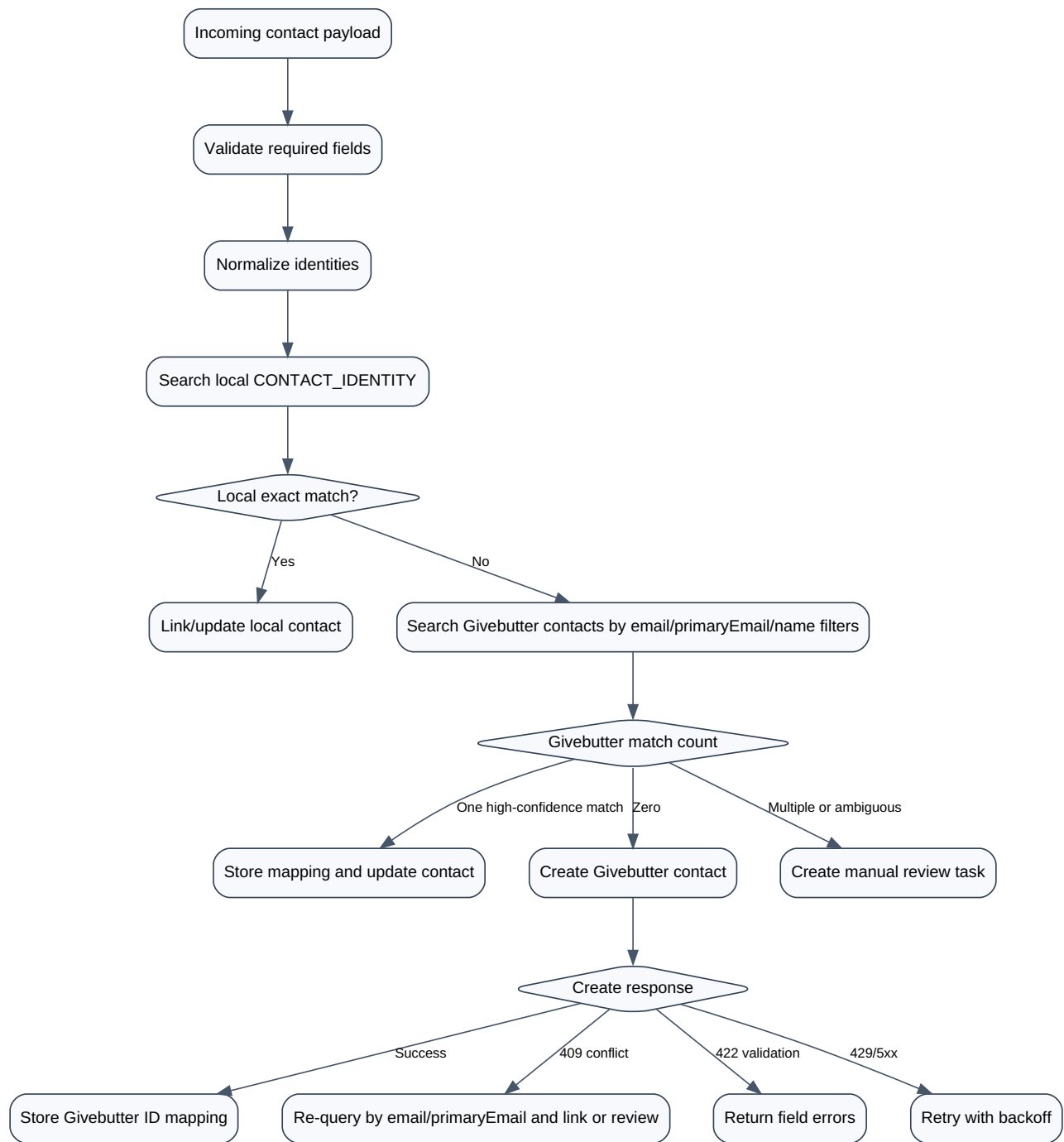


Diagram 8

11.6 Manual Review Queue

For ambiguous matches, show staff:

- Existing local contact summary.
- Candidate Givebutter contacts.

- Matching reason codes.
- Confidence score.
- Last donation date / last activity.
- Tags and household relationships.
- Actions: `link`, `create separate`, `update existing`, `ignore`, `escalate`.

11.7 Dedup Acceptance Criteria

- No API route creates a Givebutter contact without first calling the dedup service.
 - Exact email duplicate creates are idempotent.
 - Ambiguous matches do not auto-merge.
 - `409 Conflict` and `422 Validation` responses are handled explicitly.
 - The dedup job can be re-run safely without creating duplicates.
 - Staff can audit why a contact was linked, created, or sent to review.
-

12. API Client Design

12.1 Givebutter Client Responsibilities

- Centralize base URL and auth header.
- Enforce timeouts.
- Handle pagination.
- Map API errors to typed errors.
- Retry 429/5xx safely.
- Never retry non-idempotent creates unless guarded by local idempotency controls.
- Redact secrets from logs.

12.2 Suggested Internal Interface

```
interface GivebutterClient {
  listContacts(params: ContactListParams): Promise<Paginated<Contact>>;
  getContact(id: string): Promise<Contact>;
  createContact(input: CreateContactInput): Promise<Contact>;
  updateContact(id: string, input: UpdateContactInput): Promise<Contact>;
  deleteContact(id: string): Promise<void>;
  restoreContact(id: string): Promise<Contact>;

  listCampaigns(params: CampaignListParams): Promise<Paginated<Campaign>>;
  getCampaign(id: string): Promise<Campaign>;
}
```

```
listTransactions(params: TransactionListParams):  
    Promise<Paginated<Transaction>>;  
getTransaction(id: string): Promise<Transaction>;  
  
createWebhook(input: CreateWebhookInput): Promise<Webhook>;  
listWebhookActivities(webhookId: string): Promise<Paginated<WebhookActivity>>;  
}
```

13. Observability and Diagnostics

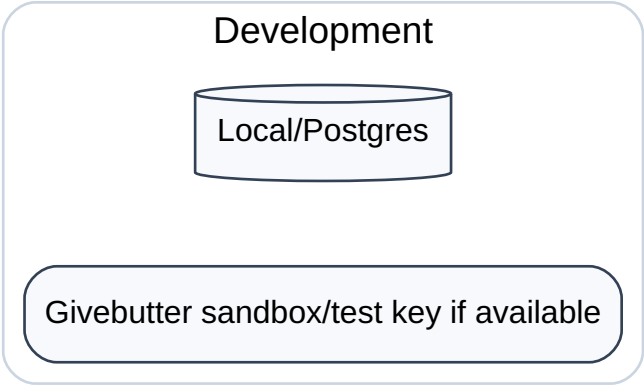
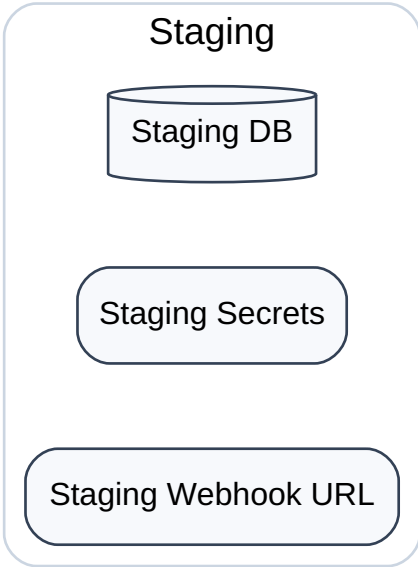
13.1 Metrics

- Givebutter API request count by endpoint/method/status.
- 429 rate-limit events.
- 401/403 auth failures.
- 409 conflicts.
- 422 validation failures by field.
- Webhook received/processed/failed counts.
- Webhook processing latency.
- Sync job retry count and dead-letter count.
- Contact dedup match rates by tier.
- Manual review queue size and age.

13.2 Admin Diagnostics Pages

Page	Purpose
API Health	Shows recent Givebutter response status, latency, 429s, and 5xxs.
Webhook Activity	Shows received events, validation status, processing state, and replay controls.
Sync Jobs	Shows queued/running/failed jobs and retry history.
Dedup Review	Allows staff to resolve ambiguous contacts.
Audit Log	Shows who changed contacts, mappings, campaigns, settings, and webhooks.

14. Deployment Environments



Each environment should use separate Givebutter API keys, separate webhook URLs, separate databases, and separate secrets.

15. Implementation Phases

Phase 1: Secure Foundation

- Backend-for-Frontend API.
- Auth provider integration.
- Secret manager integration.
- Givebutter API client.
- Basic campaign/contact/transaction read flows.
- API observability and audit logging.

Phase 2: CRM + Dedup

- Contact create/update flows.
- Dedup service and local identity tables.
- Givebutter contact mapping table.
- Manual review queue.
- Contact tags and activities.

Phase 3: Webhooks + Sync Reliability

- Webhook receiver.
- Signature verification.
- Webhook event log.
- Queue workers.
- Retry/dead-letter behavior.
- Admin diagnostics.

Phase 4: Mobile Apps

- Shared mobile design system.
- Campaign discovery.
- Donation widget/deep-link flow.
- Ticket and transaction views.
- Push notification integration.
- Offline read cache.

Phase 5: Automation and Intelligence

- Suggested duplicate resolutions.
- Donor segmentation suggestions.
- Campaign performance dashboards.
- Automated sync drift detection.
- Scheduled reports.

16. Key Risks and Mitigations

Risk	Mitigation
API key exposed in client	Never call Givebutter from browser/mobile directly; route through backend.
Duplicate contacts	Dedup before create, maintain mapping table, handle 409 conflicts, manual review.
Webhook replay or spoofing	Verify signature, store event hash/id, enforce replay window.
Rate limit issues	Queue writes, cache reads, honor <code>Retry-After</code> , cap concurrency.
Sync drift	Scheduled reconciliation jobs and admin drift reports.
Ambiguous household/person matches	Treat household association separately from person-level merge.
Mobile offline conflicts	Queue writes and require server-side conflict resolution.
Over-collection of donor PII	Collect minimum necessary fields; encrypt sensitive fields and redact logs.

17. Open Questions

1. Does the organization need donor-facing account login, staff-only tooling, or both?
2. Should Givebutter remain the sole system of record for contacts, or should the platform maintain a local golden-contact model?
3. Is there a Givebutter sandbox/test environment available for safe CI integration tests?
4. What exact webhook signature verification algorithm and headers does Givebutter require for production validation?
5. Should donations always use Givebutter widgets/hosted flows, or is a custom transaction flow required?
6. What other CRMs, email tools, accounting systems, or volunteer systems must participate in deduplication?

18. References

- Givebutter API authentication documentation: <https://docs.givebutter.com/api-reference/authentication>
- Givebutter API pagination documentation: <https://docs.givebutter.com/api-reference/pagination>
- Givebutter API rate limits documentation: <https://docs.givebutter.com/api-reference/rate-limits>
- Givebutter API errors documentation: <https://docs.givebutter.com/api-reference/errors>
- Givebutter public API documentation index: <https://docs.givebutter.com/lms.txt>
- Givebutter webhook documentation: <https://docs.givebutter.com/api-reference/webhooks/create-a-webhook>
- Givebutter contacts list documentation: <https://docs.givebutter.com/api-reference/contacts/list-all-contacts>
- Givebutter widgets documentation index: <https://docs.givebutter.com/widgets/getting-started>